Etude de cas

Mise en production sécurisée de l'application Jaydee Kanban

カンバン

Enoncé

Étude de cas KANBAN

Concepteur Développeur d'Applications – Niveau 6 – MINISTERE DU TRAVAIL DU PLEIN EMPLOI ET DE L' INSERTION
Bloc de compétences : Bloc 3 – Préparer le déploiement d'une application sécurisée
Concepteur Développeur d'Applications

Scénario

L'entreprise **Jaydee** est spécialisée dans la fabrication de pièces plastiques par injection.
Afin d'améliorer le suivi de la production et la gestion des priorités, elle a développé une application interne appelée **Jaydee Kanban**, basée sur une API **Node.js / Express** connectée à une base de données **MongoDB**.
Vous intervenez en tant que **développeur DevOps** chargé de préparer la mise en production de cette application.
L'objectif est d'en assurer la **testabilité**, la **sécurité** et la **reproductibilité** du déploiement, tout en respectant les bonnes pratiques de maintenance et de documentation.

BRIEF

Le projet consiste à :

- **analyser un code Node.js existant** afin d'en identifier les risques et la structure technique ;
- **mettre en place quelques tests d'intégration** pour garantir la fiabilité de l'API ;
- **préparer un déploiement simple et reproductible**, notamment via Docker ;
- **rédiger une stratégie CI/CD** adaptée au contexte de l'entreprise ;
- **ajouter des mesures de sécurité concrètes** dans l'application ;
- **documenter l'ensemble du processus** pour qu'un tiers puisse relancer le projet sans aide.

Instructions

Étape 1 – Analyse de l'existant

- Cloner le projet fourni et exécuter l'application localement (`npm install`, `npm run dev`, `npm test`).
- Identifier :
  - les dépendances principales (ex. Express, Mongoose) ;
  - les variables d'environnement utilisées (MONGO_URI, PORT, JWT_SECRET, etc.) ;
  - les fichiers clés du projet ;
  - la présence éventuelle de tests existants.
- Rédiger un fichier **ANALYSE_EXISTANT.md** (1 page maximum) présentant :
  - la structure du projet ;
  - les risques ou anomalies constatés ;
  - les priorités avant mise en production.

Étape 2 – Tests d'intégration

- Créer un dossier `tests/` et un fichier `columns.test.ts`.
- Utiliser **Vitest** et **Supertest** pour vérifier :
  - `GET /api/columns`
  - `POST /api/columns`
- Contrôler :
  - les codes HTTP de réponse ;
  - la structure JSON retournée ;
  - l'effet sur la base de données (insertion).
- Les tests doivent pouvoir s'exécuter via `npm test`.

Livrables :

- `tests/columns.routes.ts`
- Résultats de test valides dans la console

Étape 3 – Conteneurisation

- Créer un **Dockerfile** permettant de construire l'image Node.js de production :
  - installation des dépendances ;
  - copie du code source ;
  - lancement du serveur (`npm start`).
- Vérifier le bon fonctionnement local de l'image.
- (Optionnel) Ajouter un `docker-compose.yml` pour exécuter MongoDB dans un conteneur associé.

Livrables :

- `Dockerfile`
- Explications dans le guide de déploiement

Étape 4 – Stratégie CI/CD

- Créer un document `CI_CD_STRATEGIE.md` comprenant :
  - les objectifs de la CI/CD (automatisation, fiabilité, qualité) ;
  - les déclencheurs du pipeline (push, pull request, release) ;
  - les étapes principales (installation, test, build, déploiement) ;
  - un extrait YAML simplifié d'un pipeline (10–15 lignes maximum) ;
  - la gestion des secrets et les validations avant mise en production.

Livrable :

- `CI_CD_STRATEGIE.md` (1 page environ)

Étape 5 – Documentation du déploiement

- Rédiger un **DEPLOYMENT_GUIDE.md** décrivant :
  - les prérequis techniques (Node 20, Docker, variables d'environnement) ;
  - la procédure d'installation et d'exécution en local et dans Docker ;
  - les commandes de test ;
  - la procédure de mise à jour simple.
- Ce guide peut être fusionné avec le **README.md** final.

Livrable :

- `DEPLOYMENT_GUIDE.md` ou `README.md` complet

Étape 6 – Sécurisation et bonnes pratiques

- Identifier **deux risques techniques** (ex. accès non autorisé, abus d'API, injection).
- Implémenter **au moins un middleware de sécurité** :
  - `helmet` pour la protection des en-têtes HTTP
    ou
  - `express-rate-limit` pour limiter le nombre de requêtes.
- Activer ce middleware dans `src/app.ts`.
- Vérifier qu'une route protégée renvoie bien un code 401 sans authentification.
- Documenter les choix et limites dans `NOTE_SECURITE.md`.

Livrables :

- `src/middlewares/` (avec le middleware actif)
- `NOTE_SECURITE.md` (1 page)

Étape 7 – Synthèse et livraison

- Regrouper tous les livrables dans un dépôt Git fonctionnel.
- Rédiger un **README.md** contenant :
  - la présentation du projet et du contexte ;
  - les étapes d'installation et d'exécution ;
  - les choix techniques et mesures de sécurité ;
  - les auteurs et la version.

Livrables finaux :

- Dépôt Git complet et structuré ou projet ZIP. **Ne partagez pas le dossier node_modules !!**
- `README.md` clair et synthétique

Compétences évaluées

Domaine	Compétence	Référence
Analyse technique	Identifier la structure et les dépendances d'un projet Node.js	(Étape 1)
Tests et qualité	Mettre en place des tests d'intégration automatisés avec Vitest et Supertest	(Étape 2)
Conteneurisation	Préparer un environnement d'exécution reproductible avec Docker	(Étape 3)
CI/CD	Décrire un pipeline de déploiement continu adapté à un projet Node.js	(Étape 4)
Documentation	Rédiger une procédure de déploiement claire et autonome	(Étape 5)
Sécurité	Identifier et corriger des failles courantes d'une API Express	(Étape 6)
Communication technique	Présenter et synthétiser les choix réalisés dans une documentation claire	

Informations complémentaires et annexes

Pré-requis techniques :

- Node.js version LTS
- npm ou similaire
- MongoDB (local ou conteneur Docker)
- Docker Desktop (pour les tests de conteneurisation) ou Docker CLI
- Git pour le suivi de version

Critères de réussite :

- Les tests s'exécutent avec succès.
- L'image Docker fonctionne localement.
- Une mesure de sécurité est effectivement mise en œuvre.
- Les livrables sont clairs, complets et cohérents.